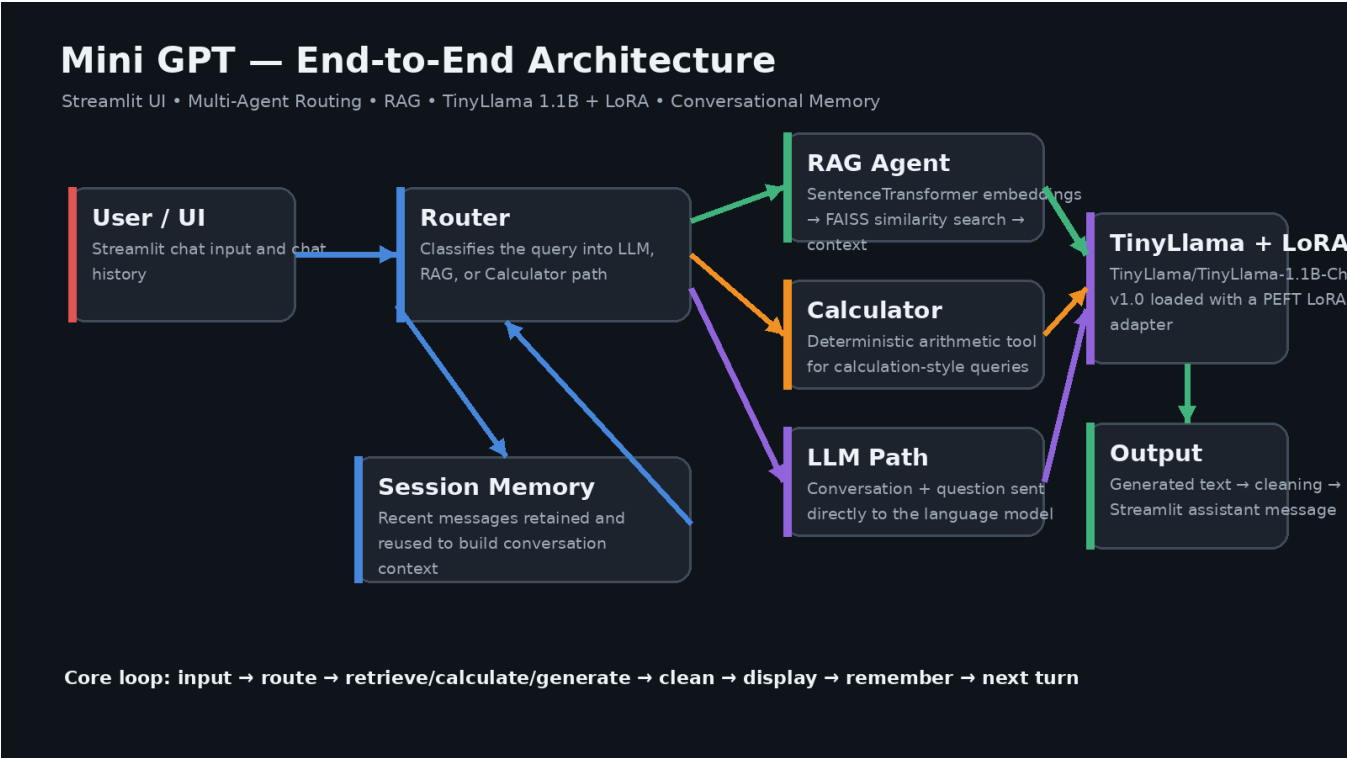


Mini GPT — Complete Project Report & Interview Preparation Guide

Technical Documentation • Architecture • Training • RAG • Debugging • Interview Preparation



Accuracy note: This report distinguishes documented project facts from values that are not preserved in the available project record. Missing training hyperparameters and dataset provenance are intentionally not invented.

Project Fact Sheet

Item	Documented detail
Project identity	Mini GPT — AI Assistant with LLM + RAG + Multi-Agent Routing
Base model	TinyLlama/TinyLlama-1.1B-Chat-v1.0
Fine-tuning	LoRA using PEFT / PeftModel
Custom training data	Approximately 14K samples, described in the project/resume as a custom dataset
RAG embedding model	SentenceTransformers: all-MiniLM-L6-v2
Vector search	FAISS, using an IndexFlatL2 index in the documented prototype
Tool agent	Calculator tool for arithmetic-style queries
Routing	Lightweight multi-agent routing based on query characteristics
Conversation memory	Streamlit session_state messages; the documented prototype builds context from recent messages
UI	Streamlit chat interface
Core Python stack	Python, NumPy, Transformers, PEFT, SentenceTransformers, FAISS, PyTorch, Streamlit
Deployment	Streamlit Community Cloud was used for an online demo
Model adapter hosting	The documented app loads the LoRA adapter from the Hugging Face Hub under the repository/model name used in the project

Values That Need Verification

Detail	What to verify
Exact dataset source/composition	The available project record confirms ~14K custom samples but does not preserve the exact collection sources or final class/category breakdown. Do not invent this in an interview.
Exact train/validation/test split	Not preserved in the available project record.
Exact epochs, learning rate, batch size, scheduler, warmup, optimizer, max sequence length	Not preserved in the available project record.
Exact GPU/CPU training hardware	Not preserved for the Mini GPT training run.
Final training metrics	No authoritative loss curve, validation metric, or benchmark table is preserved in the available record.
Final production routing implementation	The documented prototype routing code is known; later versions were modified during debugging. Verify the final app.py before quoting implementation details beyond the documented behavior.

1. Executive Summary

Mini GPT is a lightweight AI assistant designed to demonstrate how several modern LLM application techniques can be combined into one practical system. Instead of relying only on a language model, the application combines a fine-tuned TinyLlama 1.1B model, parameter-efficient LoRA adaptation, retrieval-augmented generation (RAG), a calculator tool, lightweight multi-agent routing, conversation memory, and a Streamlit chat interface.

The project was built as an engineering and learning project around the complete AI-assistant lifecycle: selecting a compact base model, adapting it with a custom dataset, integrating retrieval and tools, constructing prompts, generating responses, cleaning model output, maintaining conversational state, and exposing the system through a browser-based UI.

The most important interview point is that the project is not simply 'a chatbot'. It is a small orchestration layer around an LLM. The LLM handles language generation, RAG supplies external context, the calculator provides deterministic computation, the router selects a path, and the UI maintains the interaction.

2. Project Goal and Motivation

The primary goal was to build a compact AI assistant that demonstrates the practical integration of LLM fine-tuning, retrieval, tool use, routing, and conversational context without requiring a very large proprietary model.

The project also served as a hands-on study of the difference between model training and application engineering. Fine-tuning changes how the base model behaves, while RAG and tools extend what the application can do at inference time.

- Use a small open language model that can be loaded locally.
- Adapt the model using a custom dataset instead of training a model from scratch.
- Use LoRA/PEFT to reduce the cost of fine-tuning.
- Add RAG so the assistant can use retrieved knowledge instead of relying entirely on model parameters.
- Add a calculator path for deterministic arithmetic.
- Route different query types to different capabilities.
- Maintain recent conversational context.
- Expose the complete system through a simple Streamlit interface.

3. What the Project Actually Is

At a high level, Mini GPT is an LLM-powered orchestration application. A user enters a query in Streamlit. The application determines whether the query is better handled by the calculator, RAG, or the language-model path. For RAG questions, the query is embedded and compared with stored document embeddings through FAISS. The retrieved context is inserted into a prompt. The resulting prompt is passed to TinyLlama loaded with a LoRA adapter. The generated text is cleaned and displayed back to the user. Recent messages are retained so that later turns can include conversation context.

This architecture separates responsibilities: retrieval is not performed by the LLM itself; calculation is not delegated to the LLM; and the UI is not responsible for model internals.

4. System Architecture

The architecture can be explained in an interview from left to right: User → Streamlit UI → Router → selected capability → TinyLlama/LoRA where generation is required → output cleaning → UI → session memory.

There are three main inference paths. The RAG path retrieves relevant text before generation. The calculator path produces a deterministic result. The direct LLM path generates from conversation and the current question without retrieval.

- Presentation layer: Streamlit chat interface.
- Orchestration layer: query router.
- Knowledge layer: SentenceTransformer embeddings + FAISS vector index + documents.
- Tool layer: calculator function.
- Model layer: TinyLlama 1.1B Chat + LoRA adapter through PEFT.
- State layer: Streamlit session state for recent conversation history.
- Output layer: text cleaning and response rendering.

5. Architecture Walkthrough — Step by Step

Step 1 — Input: The user submits a question through `st.chat_input()`.

Step 2 — State update: The user message is appended to `st.session_state.messages`.

Step 3 — Context construction: The application builds a short conversation string from recent messages. In the documented prototype, the last five messages were used.

Step 4 — Routing: `route_query()` inspects the query and chooses calculator, RAG, or LLM.

Step 5A — Calculator: arithmetic-style queries are passed to `calculator_tool()`.

Step 5B — RAG: the query is embedded using all-MiniLM-L6-v2, searched in FAISS, and the top retrieved documents are joined into a context string.

Step 5C — LLM: the conversation and current question are placed into a generation prompt.

Step 6 — Generation: the selected prompt is passed through the Transformers text-generation pipeline backed by TinyLlama plus the LoRA adapter.

Step 7 — Cleaning: `clean_output()` removes known prompt artifacts such as 'Answer:' and unwanted context/instruction fragments.

Step 8 — Response: the assistant answer is stored in session state and rendered in the Streamlit chat.

Step 9 — Next turn: the stored history can be incorporated into the next prompt.

6. Tech Stack

The project deliberately uses a relatively small and accessible stack. The model and retrieval components are all Python-based, while Streamlit provides the application interface.

- Python — primary implementation language.
- PyTorch — model execution backend.
- Hugging Face Transformers — tokenizer, causal language model loading, and text-generation pipeline.
- TinyLlama 1.1B Chat — base causal language model.
- PEFT — parameter-efficient fine-tuning and LoRA adapter loading.
- SentenceTransformers — semantic embedding generation.
- FAISS — vector similarity search.
- NumPy — numerical array handling.
- Streamlit — web UI and session state.
- Hugging Face Hub — documented location for the LoRA adapter.
- GitHub — source-code repository.
- Streamlit Community Cloud — online demonstration deployment.

7. Base LLM — TinyLlama 1.1B

The base model used in the documented application is TinyLlama/TinyLlama-1.1B-Chat-v1.0. The choice reflects the project's goal of using a compact model that can be loaded and experimented with on comparatively limited hardware.

The model is a causal language model: it predicts the next token given the preceding token sequence. In the application, the Transformers pipeline performs text generation from a constructed prompt.

An important interview distinction is that TinyLlama is the base model; the project did not train a new foundation model from scratch. The project adapted the base model using LoRA.

8. Dataset — What We Know and What to Verify

The project was documented as using a custom dataset of approximately 14K samples for fine-tuning. This is the number used in the resume/project description.

The available project record does not preserve the authoritative source-by-source collection process, exact sample categories, split ratios, or final dataset statistics. Because those values are not reliably available, this report intentionally does not fabricate them.

For an interview, you should be able to answer the following from your actual training notebook or dataset file: where the samples came from, the exact number of records after cleaning, the prompt/response format, duplicate handling, train/validation split, token-length distribution, and examples of representative samples.

- Confirmed: approximately 14K custom samples.
- Not confirmed in the preserved record: exact source websites/files, exact split, exact cleaning rules, and exact category distribution.
- Interview rule: if asked for an exact value that you have not verified, say that you would check the training artifact rather than inventing it.

9. Fine-Tuning Strategy — LoRA / PEFT

The project used LoRA, or Low-Rank Adaptation, rather than updating every parameter of TinyLlama. LoRA introduces trainable low-rank matrices into selected model transformations while the original base weights remain frozen.

The practical advantage is lower trainable-parameter count, lower memory requirements, and a smaller adapter artifact. This is especially appropriate for a 1.1B parameter model when the objective is experimentation and domain/task adaptation.

The application later loads the base model first and attaches the trained adapter using `PeftModel.from_pretrained()`. This is an important implementation detail: the deployed application does not need to replace the entire base model with a newly serialized full model.

- Stage 1: Select TinyLlama 1.1B Chat as the base model.
- Stage 2: Prepare the custom ~14K training samples in the required instruction/response form.
- Stage 3: Tokenize the training examples using the base model tokenizer.
- Stage 4: Configure LoRA adapters through PEFT.
- Stage 5: Fine-tune only the LoRA parameters on the custom data.
- Stage 6: Save/publish the adapter.
- Stage 7: At inference time, load TinyLlama and attach the adapter with PEFT.

10. Fine-Tuning Details You Must Verify Before the Interview

The preserved project record does not contain authoritative values for the exact number of epochs, learning rate, batch size, gradient accumulation, optimizer, scheduler, warm-up steps, maximum sequence length, LoRA rank, LoRA alpha,

LoRA dropout, target modules, precision mode, or final training loss.

These are high-value interview details. Before the interview, open the original training notebook/script and record the exact values. Do not use generic values from tutorials as if they were the project's actual configuration.

11. RAG — Retrieval-Augmented Generation

RAG addresses a central limitation of a language model: the model's parameters are not a dynamically searchable database. Instead of asking the model to answer entirely from its learned weights, the application retrieves relevant text and gives that text to the model as context.

In the documented Mini GPT prototype, a small list of ML-related documents is embedded using SentenceTransformers. The embeddings are placed into a FAISS IndexFlatL2 index. When a query is routed to RAG, the query is embedded and the nearest vectors are searched. The top two results are joined and inserted into the prompt.

This is a compact RAG demonstration rather than a production-scale document ingestion system. It proves the retrieval → context → generation concept while keeping the implementation simple.

- Documents → embeddings.
- Embeddings → FAISS index.
- User query → query embedding.
- FAISS similarity search → top relevant documents.
- Retrieved text → prompt context.
- Prompt + context → TinyLlama.
- Generated answer → user.

12. Embedding Model — all-MiniLM-L6-v2

The documented RAG implementation uses SentenceTransformers with all-MiniLM-L6-v2. Its job is not to generate the final answer. Its job is to convert text into numerical vectors so that semantic similarity can be measured.

The same embedding model is used for the stored documents and the incoming query. This keeps the vectors in the same embedding space.

13. FAISS Vector Search

FAISS is used as the similarity-search engine. The prototype creates an IndexFlatL2 index with the embedding dimension and adds the document vectors.

For a query, the embedding is searched against the index and the nearest vectors are returned. The documented prototype retrieves the top two results.

IndexFlatL2 is simple and exact, which is appropriate for a tiny demonstration corpus. For a much larger production knowledge base, an approximate or more scalable indexing strategy could be considered.

14. Multi-Agent Routing

The project's 'multi-agent' behavior is implemented as a lightweight router rather than a framework with autonomous agents. The router chooses among three capabilities: calculator, RAG, and direct LLM generation.

This is a useful interview distinction. Calling it a 'multi-agent system' is reasonable at the application level because there are multiple specialized paths, but the implementation is deterministic routing logic rather than independent autonomous agents negotiating with one another.

- Calculator agent: deterministic arithmetic.
- RAG agent: retrieval plus LLM generation.

- LLM agent/path: direct language generation.
- Router: selects the path.

15. Calculator Tool

The calculator path exists because arithmetic is better handled deterministically than by asking a language model to estimate a numerical answer.

The documented prototype used a `calculator_tool(query)` function that attempted to evaluate the expression and returned the result as text. It also returned a calculation-error message when evaluation failed.

Important engineering caveat: using Python `eval()` directly on untrusted user input is unsafe. A production implementation should replace this with a restricted arithmetic parser or a safe expression evaluator. This is a valuable interview point because it shows awareness of the difference between a prototype and a production system.

16. Prompt Engineering and Conversation Context

The application builds prompts programmatically rather than sending only the raw user question. For RAG requests, the prompt contains an assistant instruction, recent conversation, retrieved context, the current question, and an answer marker.

For direct LLM requests, the documented prototype uses recent conversation plus the current question. The conversation history is assembled from recent messages, which gives the assistant short-term conversational continuity without implementing a full external memory database.

17. Output Cleaning

Small causal language models can reproduce parts of the prompt or output labels such as 'Answer:'. The project therefore contains a `clean_output()` step.

The documented cleaning logic removes the text after an 'Answer:' marker and removes known unwanted fragments such as 'Context:' and 'You are a helpful AI assistant', then trims whitespace.

This is a practical post-processing layer rather than a model-level correction. It improves presentation but does not guarantee factual correctness.

18. Streamlit GUI

The GUI is intentionally lightweight. Streamlit provides the page, chat input, message rendering, and session state without requiring a separate frontend framework.

The interface displays user and assistant messages using `st.chat_message()`. The application stores messages in `st.session_state.messages` so that the conversation persists during the active Streamlit session.

The project evolved through several UI debugging iterations. One observed issue was that a newly entered user message was not immediately displayed in the expected order until the next interaction. The script was revised so the current message is appended before response generation and then rendered consistently.

19. Application Initialization and Caching

The documented application uses `@st.cache_resource` for expensive model/resource initialization. This is important because loading TinyLlama, the LoRA adapter, the tokenizer, and the embedding model on every Streamlit rerun would be extremely slow.

The LLM loading function loads the base tokenizer and model, attaches the LoRA adapter, and creates a Transformers text-generation pipeline. The RAG loading function creates the embedding model, encodes the documents, builds the FAISS index, and returns the retrieval components.

20. Repository Structure

The repository evolved into a deliberately small public project. The core repository files documented during development were:

- README.md — project explanation, setup, features, demo information, and screenshots.
- app.py — main Streamlit application and inference/orchestration logic.
- requirements.txt — Python dependencies required by the application.
- .gitignore — prevents environments, caches, logs, notebook checkpoints, and Python bytecode from being committed.

21. Requirements

The documented requirements file contains the core packages used by the application: streamlit, transformers, peft, sentence-transformers, faiss-cpu, torch, and numpy.

A compatible Python environment is required. The exact Python version used for the final deployment should be verified from the environment that successfully ran the application.

22. Local Run Procedure

The practical local workflow used during development was to activate the project's conda environment, navigate to the project directory, install the requirements, and launch Streamlit.

- Create/activate the project environment.
- Navigate to the repository containing app.py.
- Install dependencies from requirements.txt.
- Run: streamlit run app.py
- Open the local Streamlit URL shown by the terminal.
- Test direct LLM, RAG, and calculator-style questions.

23. Online Deployment

The project was deployed using Streamlit Community Cloud from the GitHub repository. The deployment used the main branch and app.py as the entrypoint.

The online deployment was observed to be significantly slower than the local conda execution. This is an important real-world deployment observation. The exact bottleneck was not formally benchmarked, so it should not be presented as a proven single cause. The most plausible factors are limited cloud compute, CPU inference, cold starts, dependency/model download time, and the cost of running a 1.1B parameter model plus embedding model in the hosted environment.

24. Major Challenges and Debugging Journey

The project was not completed in one pass. Several engineering issues appeared during development and deployment.

- Model access/loading: the project used a base model plus a separate LoRA adapter, which required the correct Hugging Face model/adaptor loading sequence.
- PEFT integration: the base model had to be loaded before attaching the LoRA adapter with PeftModel.
- RAG integration: embeddings, FAISS indexing, retrieval, and prompt insertion had to work together.
- Conversation rendering: Streamlit message state and rendering order required debugging.
- Deployment performance: the application was fast locally but noticeably slower online.

- Output quality: the assistant sometimes generated incorrect or poorly grounded explanations, demonstrating that RAG and fine-tuning improve behavior but do not guarantee factual accuracy.
- Git synchronization: the repository experienced a non-fast-forward push and interactive rebase conflict while updating app.py. The conflict was resolved and the corrected commit was pushed.
- Prototype safety: the calculator's use of eval() is acceptable only as a controlled prototype and should be replaced for production use.

25. What Worked Well

The final concept successfully demonstrates a full modern LLM application pipeline rather than only a model call.

The project has a clear separation between model generation, retrieval, tools, routing, and UI.

LoRA makes the fine-tuning story technically meaningful without claiming that a foundation model was trained from scratch.

RAG gives the project an explicit grounding mechanism.

The calculator tool demonstrates deterministic tool use.

The Streamlit UI makes the project directly testable and visually demonstrable.

The GitHub repository and online demo make the project portfolio-friendly.

26. Limitations

The current system should be presented as a compact prototype/portfolio project rather than a production-grade autonomous assistant.

- Small RAG corpus in the documented prototype.
- Rule-based routing rather than learned intent classification or autonomous planning.
- Calculator implementation uses eval() in the documented prototype and is not safe for arbitrary untrusted expressions.
- No persistent database-backed memory.
- No production authentication/authorization layer.
- No formal evaluation suite or benchmark report preserved in the project record.
- Online latency was higher than local execution.
- The exact final fine-tuning hyperparameters and dataset construction details need to be verified from the original training artifacts.

27. Production-Grade Improvements

If the project were continued beyond the portfolio prototype, the architecture could be hardened without changing its fundamental idea.

- Replace eval() with a safe arithmetic parser.
- Move documents into a proper ingestion pipeline and persistent vector store.
- Add document chunking, metadata, source citations, and configurable retrieval top-k.
- Add a real intent/router layer with confidence thresholds and fallbacks.
- Use a model-serving layer instead of loading the model directly inside the Streamlit process.
- Add request logging, tracing, latency metrics, and error monitoring.
- Add automated tests for routing, retrieval, prompts, output cleaning, and UI-independent inference.
- Add model/version management for the base model and LoRA adapter.

- Benchmark CPU vs GPU inference and measure first-token/total-response latency.
- Add authentication and rate limiting for public deployment.

28. Interview — 60-Second Explanation

A strong concise explanation is: 'Mini GPT is a lightweight LLM assistant built around TinyLlama 1.1B. I fine-tuned the model using LoRA/PEFT on a custom dataset of about 14K samples. At inference time, I added a lightweight multi-agent router that decides whether a query should go to the direct LLM, a RAG pipeline, or a calculator tool. The RAG pipeline uses SentenceTransformers and FAISS to retrieve relevant context, which is then added to the prompt before generation. The whole system is wrapped in a Streamlit chat UI with short-term conversation memory, and I deployed a demo through Streamlit Community Cloud.'

29. Interview — Architecture Drawing Script

When asked to draw the architecture, draw six blocks left to right: User → Streamlit UI → Router → specialized paths → TinyLlama + LoRA → Response. Under the Router draw three branches: RAG, Calculator, and Direct LLM. Under RAG draw Documents → SentenceTransformer → FAISS → Retrieved Context. Draw Session Memory underneath the UI and connect it back to Prompt Construction.

Then explain the data flow in one sentence: 'The router decides what capability is needed, retrieval or tools provide specialized computation/context, TinyLlama performs language generation where required, and the result is cleaned, displayed, and retained for the next turn.'

30. Interview Questions You Should Be Ready For

These are the highest-value questions for this project:

- Why did you choose TinyLlama instead of a larger model?
- Why did you use LoRA instead of full fine-tuning?
- What exactly is LoRA doing mathematically?
- What is PEFT and why did you use PeftModel?
- What was in your 14K-sample dataset?
- How did you clean and format the training samples?
- What train/validation split did you use?
- What were your learning rate, epochs, batch size, and LoRA configuration?
- How did you evaluate the fine-tuned model?
- What problem does RAG solve?
- Why SentenceTransformers?
- Why all-MiniLM-L6-v2?
- What is an embedding?
- Why FAISS?
- What does IndexFlatL2 mean?
- How does similarity search work?
- Why retrieve top-k documents?
- What is the difference between fine-tuning and RAG?
- Why use a calculator tool instead of the LLM?
- How does your router decide which agent to use?

- Is your system really multi-agent?
- How does conversation memory work?
- Why use Streamlit?
- Why is the online deployment slower?
- How would you make this production-ready?
- What are the security issues with eval()?
- How would you test the RAG system?
- How would you measure hallucination?
- How would you scale the system to many concurrent users?
- How would you replace the model without changing the UI?

31. High-Risk Claims — Verify Before Saying Them

The following values should be taken directly from the original training notebook/script before an interview: exact dataset source, exact sample count if greater precision is required, train/validation/test split, epochs, learning rate, batch size, optimizer, scheduler, sequence length, LoRA rank/alpha/dropout, target modules, hardware, training duration, and final evaluation metrics.

The report intentionally distinguishes these unknowns instead of inventing them. This is important because interviewers often follow a resume bullet with a numerical question.

32. Glossary

LLM — Large Language Model; a model trained to predict/generate token sequences.

Causal LM — a language model that predicts the next token using preceding tokens.

Fine-tuning — additional training of a pretrained model on task/domain-specific examples.

LoRA — Low-Rank Adaptation; parameter-efficient fine-tuning using low-rank trainable updates.

PEFT — Parameter-Efficient Fine-Tuning; a family of techniques that reduce the amount of trainable parameters.

Embedding — a numerical vector representation of text used for semantic comparison.

RAG — Retrieval-Augmented Generation; retrieving external context before generation.

FAISS — a library for efficient vector similarity search.

Agent/tool — a specialized execution path that handles a particular task.

Routing — selecting the appropriate capability based on the incoming query.

Session state — Streamlit's per-session state mechanism used here for conversation history.

Prompt — the structured text passed to the language model to condition generation.

33. Final Mental Model

Remember the project as five layers: Model, Adaptation, Knowledge, Tools/Orchestration, and Interface.

Model = TinyLlama 1.1B. Adaptation = LoRA/PEFT trained on ~14K custom samples. Knowledge = SentenceTransformers + FAISS RAG. Tools/Orchestration = router + calculator + prompt construction + output cleaning. Interface = Streamlit + session memory + online deployment.

If you can explain those five layers, draw the architecture, explain why each technology was chosen, and honestly state the training values that you verified from your notebook, you will be able to defend the project much more confidently in an interview.

Appendix A — Core Inference Pseudocode

```
query = st.chat_input(...)

store user message in session state
build recent conversation context

agent = route_query(query)

if agent == "calculator":
    answer = calculator_tool(query)

elif agent == "rag":
    context = retrieve(query)
    prompt = conversation + context + question
    answer = TinyLlama_with_LoRA(prompt)

else:
    prompt = conversation + question
    answer = TinyLlama_with_LoRA(prompt)

answer = clean_output(answer)
store assistant answer
render assistant message
```

Appendix B — Interview Preparation Checklist

- Can I draw the architecture from memory?
- Can I explain why TinyLlama was selected?
- Can I explain LoRA versus full fine-tuning?
- Can I explain what the 14K samples contained?
- Can I state the exact training hyperparameters from my notebook?
- Can I explain how SentenceTransformers creates embeddings?
- Can I explain FAISS and IndexFlatL2?
- Can I explain the difference between RAG and fine-tuning?
- Can I explain the router and why the project is described as multi-agent?
- Can I explain the calculator tool and its security limitation?
- Can I explain Streamlit session state?
- Can I explain why cloud deployment was slower?
- Can I describe at least three real debugging problems and their fixes?
- Can I state which project details are prototype-only versus production-ready?